دانشگاه صنعتی امیرکبیر
دانشکده مهندسی برق

به نام خدا

پروژه درس مقدمه‌ای بر شبکه‌های عصبی
(پردازش تصویر به وسیله شبکه های عصبی)

پیشرفت فناوری پردازش موازی از آغاز بازی های ویدیوئی در کنار جمع آوری منابع غنی داده ها باعث شد پردازش تصویر به یکی از موضوعات جذاب در هوش مصنوعی تبدیل شود. الگوریتم های مرتبط با یادگیری عمیق و شبکه های عصبی حلقوی از گذشته توسعه داده شده بودند اما دستاورد آنها تا زمان توسعه سخت افزار های پردازش موازی مانند GPU و TPU مشخص نبود. این سه پایه ی اصلی علوم داده(داده، سخت افزار، و الگوریتم) زمینه تحقیقات بر روی پردازش تصویر را فراهم ساخت. در طول دهه گذشته الگوریتم های جالبی در این خصوص پا به عرصه ی ظهور گذاشته اند که تحلیل برخی از پیچیدگی های تصاویر را ممکن می‌سازد. توسعه و جامعیت این الگوریتم ها مستلزم معماری بلوک به بلوک و الگو های تکرار شونده، می‌باشد که به وضوح در مدل های مرسوم یادگیری عمیق قابل مشاهده می‌باشد.

این پروژه به بررسی ساختار برخی از الگوریتم های استفاده شده در شبکه های عصبی می‌پردازد و ساختار بلوک های به کار رفته درآن ها را مورد بررسی قرار می‌دهد. در فاز اول، هدف استفاده از الگو های residual و لایه ی batch normalization به عنوان تکنیک های مرسوم در پردازش تصاویر می‌باشد. در فاز دوم، با هدف قرار دادن یک مجموعه داده پزشکی به معیار های مختلف بررسی عملکرد شبکه های عصبی عمیق می‌پردازیم. سپس Heatmap های موجود در برخی از نمونه ها را بررسی می‌کنیم تا نواحی موثر در طبقه بندی آن نمونه ها را بیابیم. در فاز سوم، به کاربرد یادگیری انتقالی در حل مسائل پیشرفته تر پردازش تصویر می‌پردازیم. به عنوان مثال در این بخش با تعبیه کلمات مختلف در قالب یک جمله به توصیف تصاویر می‌پردازیم.

فاز اول

یکی از مجموعه داده های معروف مرتبط به پردازش تصویر مجموعه داده CIFAR-10 است. این داده از 10 کلاس مختلف تشکیل شده است. این مجموعه داده از دو بخش train و test تشکیل شده است و حاوی 50000 نمونه train و 10000 نمونهtest است. تصاویر این مجموعه داده resolution پایینی دارند و هر نمونه دارای ابعاد 3×32×32 می باشد. کیفیت پایین تصاویر و عدم وضوح تصاویر چالش های زیادی در حل این مسئله دنیای یادگیری ماشین، ایجاد نموده است. در این فاز، قرار است با طراحی یک شبکه عصبی عمیق، نمونه های 10 کلاس مختلف این مجموعه داده را از هم تفکیک کنیم.

1. ابتدا مجموعه داده را با استفاده از TensorFlow در سیستم خود دریافت کنید. سپس پیش پردازش های لازمه به منظور استفاده داده ها در یک شبکه عصبی با لایه های فعال ساز ReLu را به عمل آوردید.
2. با استفاده از شبکه های residual و batch normalization شبکه ای طراحی کنید.
3. معیار های ارزیابی شبکه را در طول یادگیری مشخص کنید و نمودار آن ها را برای داده های train و validation رسم کنید.
4. عوامل ایجاد over-fit در شبکه را شناسایی کنید و روش های پیشنهادی خود را برای جلوگیری از over-fit ارائه دهید. به مقایسه عملکرد شبکه پس از رفع over-fit با قبل بپردازید.
5. در نهایت پس از رفع مشکلات مرتبط با over-fit و under-fit دقت شبکه را برای نمونه های test به بیش از 80 درصد برسانید.
6. بخش امتیازی: رفتار فیلتر های مختلف مربوط به بلوک های حلقوی شبکه را بسنجید و به نمایش بکشید.

نکات:

1. بعد از پیش پردازش، داده ها را در قالب batch در GPU Memory ذخیره نمایید. تمامی مدل های شبکه های عصبی با استفاده از GPU یادگرفته شوند. (استفاده از GPU برای تسریع فرآیند یادگیری می‌باشد. کتابخانه های پردازش موازی از جمله CuDNN برای توسعه مدل های یادگیری عمیق و عملیات متعدد ماتریسی بهینه شده اند.)
2. مقدار مناسب batch را با توجه به تنوع و تعداد نمونه های مجموعه داده پیدا کنید.

3. در طراحی شبکه عمیق امکان استفاده از هیچ گونه شبکه عصبی از پیش train شده ممکن نیست(امکان استفاده از transfer learning وجود ندارد).
4. برای طراحی شبکه از Keras API استفاده نمایید.
5. استفاده از لایه Batch Normalization به همراه تابع فعال ساز ReLu الزامی است.
6. تعداد فیلتر ها را در هر لایه تنظیم کنید.
7. در طول یادگیری شبکه انتخاب تصادفی batch ممکن است باعث بهبود چشم گیر عملکرد شبکه بر روی validation در برخی بخش ها شود اما در مراحل بعدی یادگیری، تعادل وزن ها برهم بخورد و عملکرد مدل افت کند. با استفاده از توابع ModelCheckpoint در TensorFlow بهترین مدل را بر اساس loss در بین epoch های مختلف، ذخیره کنید و ارزیابی مدل بر روی داده های test را با استفاده از این مدل بسنجید.
8. دلیل استفاده از هر لایه و الگو خاص توالی لایه های مختلف در شبکه عمیق را به طور کامل در گزارش پروژه ذکر کنید.

فاز دوم

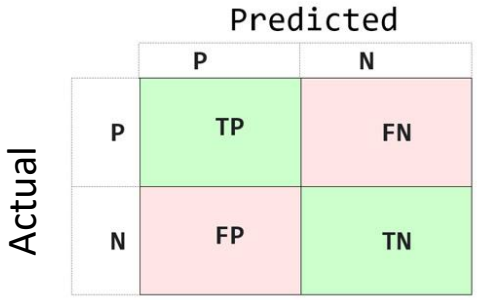
یکی از کابرد های ویژه شبکه های عصبی حلقوی، در پردازش تصاویر پزشکی می‌باشد. در این بخش از پروژه به تحلیل و تشخیص سرطان سینه با استفاده از شبکه های عصبی عمیق می‌پردازیم. پیش از مطرح کردن بخش های مختلف این فاز از پروژه، به تعریف چند معیار بررسی عملکرد شبکه های عصبی برای طبقه بندی داده ها می‌پردازیم.

ماتریس سردرگمی(Confusion Matrix)

بر اساس پیش بینی مدل برای داده هایی با کلاس باینری(همچنین قابل تعمیم به سایر کلاس های بیشتر) می‌توان چهار مفهوم تعریف کرد (داده ها با دو کلاس منفی و مثبت طبقه بندی می‌شوند):

1. True Negative (TN): تعداد نمونه هایی که به درستی منفی پیش بینی شده اند.
2. False Negative (FN): تعداد نمونه هایی که به اشتباه منفی پیش بینی شده اند (مقدار حقیقی آن ها مثبت می‌باشد.).
3. True Positive (TP): تعداد نمونه هایی که به درسی مثبت پیش بینی شده اند.
4. False Positive (FP): تعداد نمونه هایی که اشتباه مثبت پیش بینی شده اند (مقدار حقیقی آن ها منفی می‌باشد.).

این مقادیر را می‌توان به صورت ماتریسی نمایش داد:



شکل 1-ماتریس سردرگمی

باتوجه به ماتریس سردرگمی:

$$TP + FP = Predicted\ P$$
$$FN + TN = Predicted\ N$$
$$TP + FN = Actual\ P$$
$$FP + TN = Actual\ N$$

صحت، تمامیت، دقت و F_1

مفاهیم صحت (precision)، دقت(accuracy)، و تمامیت(recall این ترجمه برای مفهوم بهتر انتخاب شده است.) را نیز می‌توان به صورت زیر تعریف کرد:

$$precision = \frac{TP}{TP + FP} = \frac{TP}{Predicted\ P}$$
$$recall = \frac{TP}{TP + FN} = \frac{TP}{Actual\ P}$$
$$accuracy = \frac{TP + TN}{TP + FN + TP + FP}$$

صحت مشخص می‌کند چه کسری از میان داده هایی که مثبت پیش بینی شده اند واقعا مثبت بوده اند.

تمامیت مشخص می‌کند چه کسری از میان کل داده های مثبت درست پیش بینی شده اند (تمامیت پیش بینی های مثبت).

دقت مشخص می‌کند چه کسری از کل داده ها درست پیش بینی شده اند.

نکته: در داده های پزشکی مقادیر FP در مراحل بعد مشخص می‌شود که تشخیص به اشتباه مثبت بوده است (با آزمایش های فراوان)، اما مقادیر FN دیگر مورد بررسی قرار نمی‌گیرد (بیمار به اشتباه سالم تلقی شده است.) پس این مقدار بسیار برای کاربرد های پزشکی حائز اهمیت است. در نتیجه در این بخش مقدار recall از اهمیت زیادی برخوردار است.

برای ایجاد تعادلی میان دو تعریف recall و precision مفهومی به عنوان معیار F_1 را به صورت زیر تعریف می‌کنیم که میانگین همساز(Harmonic mean) از این دو مقدار می‌باشد:

$$F_1 = \frac{2}{\frac{1}{recall} + \frac{1}{precision}} = \frac{2.precision.recall}{preicision + recall}$$

نمودار Receiver Operating Characteristic(ROC)

برای مقایسه عملکرد مدل نسبت به مقادیر مختلف recall و precision می‌توان از نمودار ROC استفاده کرد.

برای تعریف این نمودار چند معیار عملکرد دیگر نیز تعریف می‌کنیم:

نرخ TP(TPR) همان recall است و نرخ FP(FPR) نرخ مقادیر مقادیر منفی ای می‌باشد که به اشتباه مثبت پیش بینی شده اند:

$$TPR(recall) = \frac{TP}{Actul\ P} = sensivity,$$

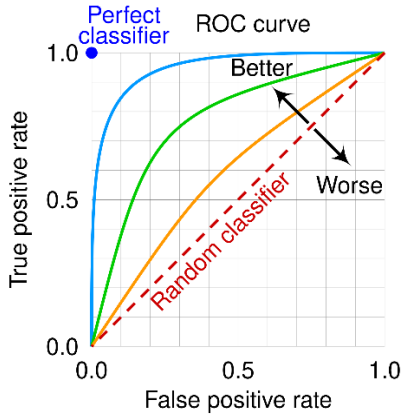
$$FPR = \frac{FP}{Actual\ N} = 1 - \frac{TN}{Actual\ N} = 1 - TNR = 1 - specificity$$

TPR حساسیت و TNR را $specificity$ نیز می‌گوییم.

نمودار ROC به این صورت است که نرخ TPR را بر حسب FPR محاسبه می‌کند. به عبارت دیگر حساسیت مدل را بر حسب $1 - specificity$ ترسیم می‌کند.

هرچه مدل TPR بیشتری داشته باشد همراه با اینکه FPR آن کمتر باشد، مدل بهتری خواهیم داشت. پس بهترین مدل طبقه بندی کننده FPR=0 و TPR=1 را دارا خواهد بود.

هر طبقه بندی را می‌توان به صورت جداسازی بردار های مختلف نمونه های مجموعه داده در یک فضای n بعدی مدل سازی کرد. با تعریف آستانه(threshold) های مختلف می‌توان مقادیر مختلف TPR و FPR را محاسبه کرد و آن ها را رسم کرد. به مساحت زیر نمودار ROC، مساحت زیر نمودار(Area Under the Curve) می‌گوییم. قاعدتا هرچه این مقدار به یک نزدیک تر باشد جداکننده بهتری خواهیم داشت. برای محاسبه این مقادیر از توابع کتابخانه scikit-learn استفاده کنید.



شکل 2- نمودار ROC

بخش های فاز دوم

1. ابتدا مجموعه داده مربوطه را با استفاده از سیستم خود(یا با استفاده از سرور های Google Colab و Kaggle) دریافت کنید. سپس پیش پردازش های لازمه را به عمل آورید. دیتا را به 3 دسته، train validation و test تقسیم کنید. در صورت نیاز از standardization استفاده کنید.
2. در خصوص balanced یا unbalanced بودن مجموعه داده بحث کنید.
3. با استفاده از انواع مختلف لایه های حلقوی، یک شبکه عصبی طراحی کنید که دارای $f_1 score$ بیشتر از 90 درصد بر روی داده های test باشد.
4. نمودار های loss، accuracy، precision، و recall را بر حسب محور epoch برای داده های train و validation رسم کنید و رفتار شبکه عصبی خود را در هر نمودار تحلیل کنید.
5. معیار های f_1 ، $precision$ ، $recall$ و AUC را برای داده های test در طول یادگیری محاسبه کنید. در خصوص تفاوت این معیار ها بحث کنید و بهترین معیار را در این کاربرد خاص مشخص کنید.
6. ترتیب لایه ها و معماری شبکه نهایی را با استفاده از plot_model به نمایش بکشید.
7. بخش امتیازی: Heatmap مربوط به یک نمونه با تشخیص مثبت و یک نمونه با تشخیص منفی را رسم کنید و پاسخ شبکه را با توجه به مقدار فعال سازی دسته بندی مربوطه توجیه کنید. Heatmap بخش هایی از تصویر هستند که بیشترین تاثیر را در پیش بینی نهایی مدل داشته اند. در این کاربرد، بافت هایی از تصاویر که باعث تشخیص تصویر به عنوان نمونه سرطانی یا غیر سرطانی شده اند، می‌توان به صورت Heatmap در نظر گرفت.

نکات:

1. از طریق لینک زیر به مجموعه داده دسترسی پیدا کنید:

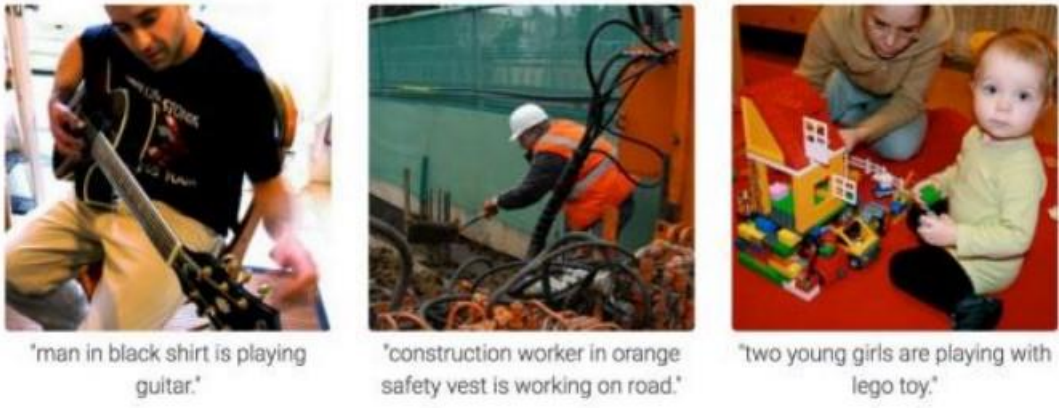
Breast Histopathology Images (kaggle.com)

2. بعد از پیش پردازش، دیتا را در قالب batch در GPU Memory ذخیر نمایید. تمامی مدل های شبکه عصبی با استفاده از GPU یادگرفته شوند.
3. مقدار مناسب batch را با توجه به تنوع و تعداد نمونه های دیتا پیدا کنید.
4. در طراحی شبکه عمیق امکان استفاده از هیچ گونه شبکه عصبی از پیش آموزش دیده ممکن نیست(امکان استفاده از transfer learning وجود ندارد).

5. برای طراحی شبکه از Keras API استفاده نمایید.
6. استفاده از لایه Batch Normalization به همراه تابع فعال ساز ReLu الزامی است.
7. تعداد فیلتر ها را در هر لایه tune کنید.
8. در طول یادگیری شبکه انتخاب تصادفی batch ممکن است باعث بهبود چشم گیر عملکرد شبکه بر روی validation در برخی بخش ها شود اما در مراحل بعدی یادگیری، تعادل وزن ها برهم بخورد و عملکرد مدل افت کند. با استفاده از توابع ModelCheckpoint در TensorFlow بهترین مدل را بر اساس loss در بین epoch های مختلف، ذخیره کنید و ارزیابی مدل بر روی داده های test را با استفاده از این مدل بسنجید.
9. دلیل استفاده از هر لایه و الگو خاص توالی لایه های مختلف در شبکه عمیق را به طور کامل در گزارش پروژه ذکر کنید.

فاز سوم: توصیف عکس(Image Captioning)

یکی از حوزه های جذاب در یادگیری ماشین، توصیف یک عکس با یک جمله است. در واقع هدف ایجاد و آموزش مدلی است که بتواند یک تصویر را به عنوان ورودی بگیرد و در نهایت یک جمله در توصیف آن عکس در خروجی خود تولید کند. تصویر زیر نمونه ای از خروجی این شبکه را نشان می دهد.



شکل 3 - خروجی مطلوب یک شبکه آموزش دیده برای توصیف تصویر

حال در این پروژه قصد داریم یک مدل برای رسیدن به این هدف پیاده سازی نماییم. ساختار کلی این مدلها به این صورت است که یک شبکه CNN جهت تولید ویژگی های تصاویر وجود دارد و در کنار آن روش های مختلفی برای Embedding جملات موجود است که در نهایت بردار ویژگی تصاویر و متن در کنار هم قرار گرفته و به عنوان ورودی یک شبکه بازگشتی اعمال می شود تا در نهایت جمله نهایی را تولید نماید. در ادامه بیشتر با بخش های مختلف آن آشنا خواهید شد.

دادگان

مجموعه دادگانی پیشنهادی که شما در این پروژه می‌توانید از آنها استفاده کنید در جدول زیر آمده است.

Flickr8k	Contains 8K images with 5 captions each split : 7k images for training and 1k images for validation	https://www.kaggle.com/datasets/adityajn105/flickr8k
Flickr30k	Contains 30K images with 5 captions each split : 28K images for Training and 2k images for validation	https://www.kaggle.com/datasets/hsankesara/flickr-image-dataset
MSCOCO	Contains 120K images with 5 captions for each split : 80k images for Training and 40k images for Validation	https://www.kaggle.com/datasets/nikhil7280/coco-image-caption

با آماده سازی تصاویر برای اعمال به شبکه های کانولوشنی پیشتر آشنا شدید. در اینجا جملات نیز باید پیش پردازش شوند تا به بردارهایی از اعداد تبدیل شوند. ما در اینجا برای سادگی پیشنهاد می کنیم که از لایه‌ی Embeddingدر پایتورچ استفاده کنید که نحوه کار با این لایه را در پیوند زیر مشاهده می کنید(شما میتوانید از سایر روش ها هم به انتخاب خودتان بهره ببرید که نیاز هست که در گزارش خودتان به آن اشاره کنید):

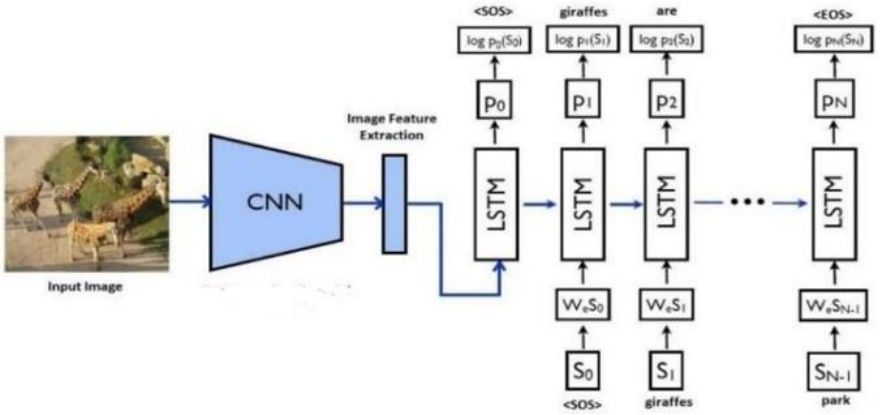
Embedding — PyTorch 2.5 documentation

پارامتری به نام Embedding-dim در آن وجود دارد که می توانید آن را ۳۰۰ در نظر بگیرید که البته انتخاب آن در اختیار شما می باشد که در واقع این عدد مشخص می کند که برای هر کلمه یک بردار عددی با طول ۳۰۰ در نظر بگیرید. نکته که مهمی که در پیش پردازش داده ها باید توجه نمایید، این است که باید برای هر جمله از توکن های شروع و پایان جمله < SOS>و <EOS>استفاده نماییم. که توکن های خاصی می باشد که توسط خود شما تعریف می شوند. همچنین باید مجموعه لغات موجود در مجموعه دادگان خود را پردازش و به هر کدام از آنها یک Index نسبت دهید. بهتر است علامت های نگارشی از جملات حذف شوند. همچنین از آنجایی که

جملات Caption ها طولهای متفاوتی دارند باید طول آن ها باهم یکسان شوند، که این کار را با Padding مناسب می توانید انجام دهید که می توان یک طول مشخص ثابت را در نظر گرفت یا یکسان‌سازی را در هر batch miniانجام داد.

آموزش مدل

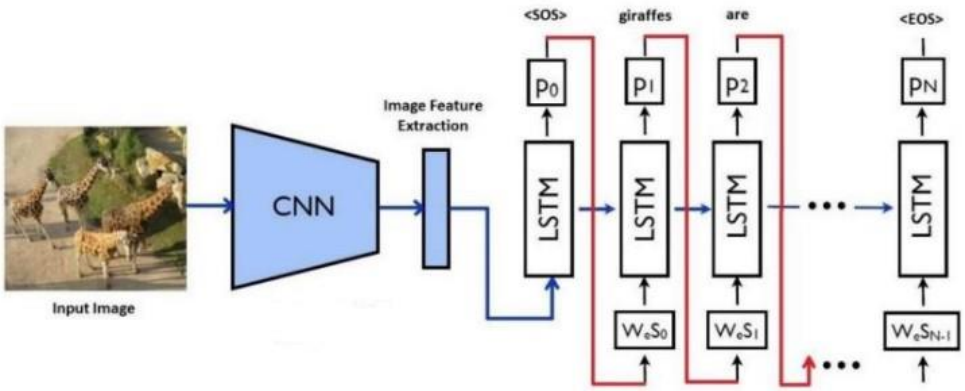
در شکل 4مدل کلی مد نظر را مشاهده می کنید. همانطور که مشاهده می کنید، بخشی از مدل جهت استخراج ویژگی تصاویر مورد استفاده قرار می گیرد. در آخرین لایه شبکه کانولوشنی ویژگی های تصویر استخراج می شود که در نهایت نیاز است به یک لایه خطی جهت استخراج ویژگی های مورد نظر با ابعاد مناسب جهت ورود به شبکه بازگشتی، استفاده نمود. در این قسمت از یک لایه شبکه LSTM (یا هر شبکه RNN دیگر مانند GRU و BiLSTM) استفاده می نماییم و بردار های Embed شده جملات در کنار بردار تصویر به آن داده شده و خروجی آن به یک لایه خطی به سایز ورودی Hidden state و سایز خروجی تعداد کلمات موجود در مجموعه دادگان اعمال می شود و به این ترتیب به محاسبه خطا و پیشبینی مدل می پردازیم.



شکل 4 - مدل پیشنهادی

پیش بینی شبکه

بعد از آموزش شبکه، نیاز دارید تا شبکه را ارزیابی نمایید. جهت ارزیابی شبکه باید به صورتی که در شکل 5نشان داده شده از شبکه استفاده نماییم. همانطور که می دانیم در زمان تست شبکه آموزش داده شده، Caption وجود ندارد و ما باید برای یک تصویر Caption تولید نماییم. برای این منظور روش های مختلفی وجود دارد ولی ما در اینجا مدل پایین را پیشنهاد می دهیم. در یک تابع به عنوان ورودی، تصویر تست و مدل آموزش داده شده را جهت پیش بینی کلمات اعمال می کنیم.



شکل 5 - نحوه استفاده از مدل در هنگام پیش بینی

همانطور که می‌دانید برای استخراج ویژگی مناسب از تصاویر نیاز به بکارگیری از یک شبکه عمیق می‌باشد که با توجه به محدودیت در دادگان امکان آموزش مدل CNN بسیار عمیق ممکن نمی‌باشد. شما می‌توانید برای استفاده از شبکه‌های عمیق مانند Resnet، Inception، MobileNet و ... از یادگیری انتقالی (transfer learning) استفاده کنید. برای اینکار شما می‌توانید از شبکه‌های آموزش دیده شده با دادگانImageNet که در PyTorch وجود دارند استفاده کنید. برای اینکار می‌توانید از fine-tune کردن لایه‌های آخر نیز استفاده کنید.

بخش امتیازی: در قسمت RNN شبکه نیز می‌توانید از مدل های آموزش دیده شده مانند BERT که در NLP مورد استفاده قرار می‌گیرند استفاده کنید.

توضیحات:

- توضیحات و لینک‌های دیتاست‌های مورد نیاز برای انجام پروژه در متن آمده است. متن پروژه را به دقت مطالعه کنید و خواسته‌های هر فاز را انجام دهید.
- فایل تحویل پروژه می‌بایست یک فایل زیپ شامل فایل word و pdf گزارش پروژه به همراه تمام کدهای اجرا شده و به فرمت ipynb یا py و مراجع مورد استفاده باشد.
- مدل‌های آموزش دیده شده را ذخیره کنید. در روز ارائه عملکرد مدل‌ها تست خواهند شد. (برای مثال در فاز سوم از شما خواسته خواهد شد یک عکس به شبکه وارد و توصیف عکس را ارائه کنید.)